

Machine Learning

Lecture 2: From Representations to Decisions

A textbook treatment

Based on the lecture notes and slides of Mounir Ghogho

September 25, 2026

Contents

How to read this book	2
2 From Representations to Decisions	3
How to read the formulas in this chapter	3
2.1 From Objects to Numerical Representations	4
2.2 Decision Making When the Distribution Is Known	6
2.3 From Known Distributions to Finite Data	13
2.4 Parameterized Models	18
2.5 Learning the Parameters from Data	20
2.6 Evaluating a Learned Model	26
2.7 The Learning Problem in One Picture	29
Notes on the original slides and notes	30
Chapter takeaways	31

How to read this book

Each concept in this book is presented in the same rhythm.

- 1. Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
- 2. Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
- 3. Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
- 4. Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
- 5. Examples.** From “trivially small” to “genuinely useful.”
- 6. Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, “say it out loud” version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Basic probability (random variables, conditional distributions, expectations), vectors in $\mathbb{R}^d$, and a first course in calculus. The information-theoretic quantities used (entropy, cross-entropy, Kullback–Leibler divergence) are defined and proved from scratch where they appear.

Chapter 2

From Representations to Decisions

How to read the formulas in this chapter

This chapter is where the mathematics starts, and the slides can look intimidating. Two things make it manageable. First, there are only a handful of symbols, listed below; every formula in the chapter is built from them. Second, every formula is followed by a *Reading the formula* box that says the same thing in words and then plugs in small numbers. If a formula looks frightening, skip to its box, then come back.

Symbol	Meaning in words
$x \in \mathcal{X} \subseteq \mathbb{R}^d$	one input, written as a list of d numbers (the <i>features</i>)
$y \in \mathcal{Y}$	the output we want to predict (a number, or a class label)
X, Y	the input and output regarded as <i>random</i> : “a house drawn at random and its price”
$\mathbb{P}(X, Y)$	the joint distribution: how likely each (input, output) pair is in the world
$\mathbb{P}(Y \mid X = x)$	the distribution of the output <i>once we know</i> the input equals x
$\mathbb{E}[\cdot \mid X = x]$	an average over that conditional distribution
$a \in \mathcal{A}$	an <i>action</i> we might take (a predicted number, a chosen class, “raise the alarm”)
$\ell(y, a)$	a <i>loss</i> : how much it costs to take action a when the truth is y
$f^*(x)$	the best possible decision rule when the distribution is known (the <i>Bayes rule</i>)
$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$	the training data: N examples; superscript (i) = example number
$x_j^{(i)}$	feature number j of example number i
$f_\theta, \theta \in \Theta$	a model with adjustable numbers θ (the <i>parameters</i>)
$\mathcal{H} = \{f_\theta : \theta \in \Theta\}$	the <i>hypothesis space</i> : all the functions the model can represent
$R(\theta)$	the <i>population risk</i> : average loss over the whole world (unknowable)
$\hat{R}(\theta)$	the <i>empirical risk</i> : average loss over the examples we have
λ	a <i>hyperparameter</i> : a setting chosen by us, not learned (e.g. k in kNN)
$\mathbf{1}(\text{condition})$	the <i>indicator</i> : equals 1 if the condition holds, 0 otherwise
$\arg \min_a g(a)$	the value of a that makes $g(a)$ smallest (“the best a ”)
Δ^{C-1}	the set of probability vectors over C classes (nonnegative, sum to 1)

2.1 From Objects to Numerical Representations

What are we trying to get?

A learning algorithm cannot receive a house, only numbers. We want to know how an object becomes a vector x , which choices are hidden in that step, and why one careless choice (coding categories as integers) can silently break everything that follows.

Describe, then encode

Chapter 1 described learning as acquiring, through experience, the ability to make appropriate decisions in new situations. A mathematical learning problem begins by specifying the information available for those decisions. Properties of the object are first *selected* to describe it and then *encoded* mathematically:

$$\text{object} \longrightarrow \text{features describing it} \longrightarrow \text{representation } x \in \mathbb{R}^d.$$

Definition 2.1.1 (Input space, feature vector). Once an object is described by d numerical components, we write

$$x = [x_1, x_2, \dots, x_d]^\top \in \mathcal{X} \subseteq \mathbb{R}^d,$$

where $\mathcal{X}$ is the *input space*. Components may be of different kinds: a nor-

Two steps, both made by humans: choose what to describe, then choose how to turn the description into numbers.

malised measurement in $[0, 1]$, a binary feature in $\{0, 1\}$, or a categorical feature spread over several binary components (*one-hot encoding*: a category among K possibilities becomes a vector of length K with a single 1 in the position of the category).

Reading the formula

$x = [x_1, \dots, x_d]^\top$ is just “a list of d numbers, written as a column.” For a house described by area 145 m², 4 rooms, age 12 years and neighbourhood A out of $\{A, B, C\}$:

$$x = [145, 4, 12, \underbrace{1, 0, 0}_{\text{neighbourhood } A}]^\top \in \mathbb{R}^6.$$

Here $d = 6$: three numerical features and three binary components for one categorical feature. A house in neighbourhood C would end with 0, 0, 1.

Example 2.1.2 (Why not code neighbourhoods as 1, 2, 3?). Suppose neighbourhoods A, B, C are coded as a single number 1, 2, 3. Then, in any method that measures similarity by distance (Section 2.3), a house in A is at distance 1 from a house in B but at distance 2 from one in C : the coding has invented an ordering and a notion of “ B is halfway between A and C ” that has no meaning. With one-hot coding the three neighbourhoods are pairwise at the same distance $\sqrt{2}$: $\|(1, 0, 0) - (0, 1, 0)\| = \sqrt{1 + 1}$. The encoding step is not a formality; it decides what “similar” means.

The representation controls the available information

A house described only by its area offers less information for predicting price than one described by area, location, age, condition and number of rooms. Additional components help when they expose information relevant to the decision; irrelevant, redundant or unreliable features increase d without making the representation more informative. Images, audio and text make this visible: pixel values provide numbers, but shape, object identity and meaning are not explicitly encoded in them. Traditional machine learning relied on human-designed features; modern methods can learn representations from data (Section 1.5).

Example 2.1.3 (Three descriptions of the same email). Description 1: the length in characters ($d = 1$). Description 2: counts of 10,000 words ($d = 10,000$). Description 3: the same counts plus 500 random numbers ($d = 10,500$). Description 2 contains everything needed to detect most spam; description 1 does not; description 3 contains the same information as 2 but forces the learner to discover that 500 coordinates are noise, which costs data. More features are not automatically better.

Gotcha

- Integer codes for categories create fake distances (Example 2.1.2). Use one-hot coding unless the categories are genuinely ordered (“small, medium, large”).
- Units matter. A feature in metres and the same feature in millimetres are

the same information but behave very differently in any distance-based method (Section 2.3).

Summary: Numerical representations

- Object $\rightarrow$ chosen features $\rightarrow$ vector $x \in \mathbb{R}^d$. Both arrows are human choices.
- Categorical features become one-hot blocks; integer codes invent spurious order.
- The representation bounds what any later stage can learn.

2.2 Decision Making When the Distribution Is Known

What are we trying to get?

Before worrying about data, we ask an easier question: *if we knew everything about the world*, what would the best decision be? The answer, the Bayes decision rule, is the target that every learning method tries to approximate, and it already tells us that “the best prediction” depends on what mistakes cost.

Uncertainty that remains after seeing the input

Let the random vector X represent the input and the random variable Y the output of interest. Their joint distribution $\mathbb{P}(X, Y)$ describes which pairs may occur and how likely they are. Once the input takes the value x , the conditional distribution $\mathbb{P}(Y \mid X = x)$ summarises the remaining uncertainty about the output. Houses with the same recorded description may still sell at different prices because relevant information is missing, measurements are imperfect, or market conditions vary. The conditional distribution accounts for this instead of forcing one output onto every input.

Even with a perfect description, two houses can sell at different prices. The conditional distribution is the honest way to say “we know a lot but not everything.”

Reading the formula

$\mathbb{P}(Y \mid X = x)$ reads “the probabilities of the possible outputs, *given that* the input is x .” Concretely, for a 145 m² house in neighbourhood A , it might say: price 1.0 million with probability 0.5, 1.2 million with probability 0.3, 1.6 million with probability 0.2. All three prices are possible for that same description. Nothing is being predicted yet; this is a description of the world.

Actions and their consequences

Knowing the possible outputs and their probabilities does not by itself say what to do. That also depends on the consequences of each action.

Definition 2.2.1 (Decision loss). Let $\mathcal{A}$ be the *action space*, the set of choices available to the decision maker. If the true output is y and the action $a \in \mathcal{A}$ is chosen, the *decision loss*

$$\ell_{\text{dec}}(y, a)$$

is the cost assigned to that outcome. In regression, $\mathcal{A}$ may be the set of

numerical predictions; in classification, the set of class labels. Keeping a distinct from y is useful because decisions and outcomes need not be the same kind of object.

Reading the formula

$\ell_{\text{dec}}(y, a)$ is a table (or formula) with two inputs: what actually happened (y) and what you did (a). It returns a number: the damage. Examples:

- Squared error, $\ell_{\text{dec}}(y, a) = (y - a)^2$: predicting $a = 200$ when $y = 230$ costs $30^2 = 900$.
- Absolute error, $|y - a|$: the same mistake costs 30.
- Zero-one loss, $\mathbf{1}(y \neq a)$: any wrong class costs 1, a right one costs 0.
- A cost table for a medical decision: rows are the true condition, columns the action, entries the cost. Deciding “discharge” when the truth is “severe” might cost 20; “treat” when “healthy” might cost 3.

The Bayes decision rule

For a given input, different outputs may still occur. We therefore choose the action with the smallest *expected* loss under the conditional distribution.

Definition 2.2.2 (Bayes decision rule). For each input x , the *Bayes decision rule* chooses

$$f^*(x) \in \arg \min_{a \in \mathcal{A}} \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x].$$

The quantity minimised, $\mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x]$, is the *conditional risk* of action a at input x . The word “Bayes” refers to optimal decision making under a probability distribution; it does not require the parameters of any model to have prior distributions.

Reading the formula

Read it in three steps, for one fixed x .

1. List the candidate actions a .
2. For each a , compute the *average damage*: go through every possible output y , multiply its loss $\ell_{\text{dec}}(y, a)$ by its probability $\mathbb{P}(Y = y \mid X = x)$, and add up. That is $\mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x]$.
3. Pick the a with the smallest average damage. That is $\arg \min_a$.

With the house above (1.0, 1.2, 1.6 million with probabilities 0.5, 0.3, 0.2) and squared loss, the average damage of predicting $a = 1.2$ is $0.5(0.2)^2 + 0.3(0)^2 + 0.2(0.4)^2 = 0.02 + 0 + 0.032 = 0.052$. Try $a = 1.16$: $0.5(0.16)^2 + 0.3(0.04)^2 + 0.2(0.44)^2 = 0.0128 + 0.00048 + 0.03872 = 0.052$; the two are equal to three decimals, and Theorem 2.2.4 will tell us the exact best value is $a = 1.18$.

The rule minimises *pointwise*, one x at a time. The next theorem confirms that doing the best thing at every x is the best thing overall.

Theorem 2.2.3 (Pointwise optimality is global optimality). *Let $f : \mathcal{X} \rightarrow \mathcal{A}$ be any decision rule and f^* the Bayes rule of Definition 2.2.2 (assume the minimum exists at every x and all expectations are finite). Then*

$$\mathbb{E}[\ell_{\text{dec}}(Y, f^*(X))] \leq \mathbb{E}[\ell_{\text{dec}}(Y, f(X))].$$

The left side is called the Bayes risk; no rule has smaller expected loss.

Proof. By the tower property of conditional expectation, $\mathbb{E}[\ell_{\text{dec}}(Y, f(X))] = \mathbb{E}[\mathbb{E}[\ell_{\text{dec}}(Y, f(X)) \mid X]]$. For every value x , $f(x)$ is one particular action, so by the definition of $f^*(x)$ as a minimiser,

$$\begin{aligned} \mathbb{E}[\ell_{\text{dec}}(Y, f(x)) \mid X = x] &\geq \min_a \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x] \\ &= \mathbb{E}[\ell_{\text{dec}}(Y, f^*(x)) \mid X = x]. \end{aligned}$$

Averaging this inequality over x (taking $\mathbb{E}$ over X) preserves it, giving the claim. $\square$

Key insight

An expectation over pairs (X, Y) can be computed as “first average over Y for each fixed x , then average over x .” Because the inner averages are minimised separately at each x , the outer average is minimised too. This is why we may think about one input at a time.

Regression: which number to report

Theorem 2.2.4 (Squared loss selects the conditional mean). *Let $\ell_{\text{dec}}(y, a) = (y - a)^2$ and suppose $\mathbb{E}[Y^2 \mid X = x] < \infty$. Write $m(x) = \mathbb{E}[Y \mid X = x]$. Then for every action a ,*

$$\mathbb{E}[(Y - a)^2 \mid X = x] = \text{Var}(Y \mid X = x) + (m(x) - a)^2,$$

so the conditional risk is minimised uniquely at $a = m(x)$: $f^(x) = \mathbb{E}[Y \mid X = x]$.*

Proof. Write $Y - a = (Y - m(x)) + (m(x) - a)$ and square:

$$(Y - a)^2 = (Y - m(x))^2 + 2(Y - m(x))(m(x) - a) + (m(x) - a)^2.$$

Take conditional expectations given $X = x$. The first term gives $\text{Var}(Y \mid X = x)$. In the middle term, $(m(x) - a)$ is a constant (it does not involve Y), and $\mathbb{E}[Y - m(x) \mid X = x] = m(x) - m(x) = 0$, so the middle term vanishes. The last term is constant. Hence the formula. The variance does not depend on a , and $(m(x) - a)^2 \geq 0$ with equality only at $a = m(x)$. $\square$

Reading the formula

The formula says: *average squared error = unavoidable spread + your own offset squared*. The spread $\text{Var}(Y \mid X = x)$ is there whatever you do: it is the variability of prices among houses with this description. The only part you control is how far your number a is from the conditional mean $m(x)$.

House example: outcomes 1.0, 1.2, 1.6 with probabilities 0.5, 0.3, 0.2. Condi-

tional mean $m = 0.5 + 0.36 + 0.32 = 1.18$. Variance $= 0.5(0.18)^2 + 0.3(0.02)^2 + 0.2(0.42)^2 = 0.0162 + 0.00012 + 0.03528 = 0.0516$. Predicting $a = 1.2$: risk $= 0.0516 + (0.02)^2 = 0.0520$, which is what we computed by hand above. Predicting $a = 1.18$: risk $= 0.0516$, the floor.

Key insight

Add and subtract the mean, then square. The cross term dies because the mean is exactly the number that makes $\mathbb{E}[Y - m] = 0$. This “complete the square around the mean” move is the single most reused computation in statistics.

The loss determines *which summary* of the conditional distribution should be reported.

Proposition 2.2.5 (Absolute loss selects a conditional median). *Let $\ell_{\text{dec}}(y, a) = |y - a|$ and suppose $\mathbb{E}[|Y| \mid X = x] < \infty$. Any conditional median m , i.e. any number with $\mathbb{P}(Y \leq m \mid X = x) \geq \frac{1}{2}$ and $\mathbb{P}(Y \geq m \mid X = x) \geq \frac{1}{2}$, minimises $\mathbb{E}[|Y - a| \mid X = x]$ over a .*

Proof. All probabilities and expectations below are conditional on $X = x$; we drop the conditioning to lighten notation. Let m be a median and $a > m$. For each possible value y , compare the two losses:

- if $y \leq m$: $|y - a| - |y - m| = (a - y) - (m - y) = a - m$;
- if $y \geq a$: $|y - a| - |y - m| = (y - a) - (y - m) = -(a - m)$;
- if $m < y < a$: $|y - a| - |y - m| = (a - y) - (y - m) = a + m - 2y > a + m - 2a = -(a - m)$.

In all cases $|y - a| - |y - m| \geq (a - m)[\mathbf{1}(y \leq m) - \mathbf{1}(y > m)]$. Taking expectations,

$$\begin{aligned} \mathbb{E}|Y - a| - \mathbb{E}|Y - m| &\geq (a - m)[\mathbb{P}(Y \leq m) - \mathbb{P}(Y > m)] \\ &= (a - m)[2\mathbb{P}(Y \leq m) - 1] \geq 0, \end{aligned}$$

because $\mathbb{P}(Y \leq m) \geq \frac{1}{2}$. The case $a < m$ is symmetric, using $\mathbb{P}(Y \geq m) \geq \frac{1}{2}$. So no a beats m . $\square$

Proposition 2.2.6 (Asymmetric absolute loss selects a conditional quantile). *Fix $\tau \in (0, 1)$ and let $\ell_\tau(y, a) = \tau(y - a)$ if $y \geq a$ and $(1 - \tau)(a - y)$ if $y < a$ (under-prediction costs τ per unit, over-prediction $1 - \tau$ per unit). Any conditional τ -quantile m , i.e. any m with $\mathbb{P}(Y \leq m \mid x) \geq \tau$ and $\mathbb{P}(Y \geq m \mid x) \geq 1 - \tau$, minimises $\mathbb{E}[\ell_\tau(Y, a) \mid x]$.*

Proof. Same three cases as in Proposition 2.2.5, for $a > m$: if $y \leq m$ the difference $\ell_\tau(y, a) - \ell_\tau(y, m)$ equals $(1 - \tau)(a - m)$; if $y \geq a$ it equals $-\tau(a - m)$; if $m < y < a$ it equals $(1 - \tau)(a - y) - \tau(y - m) \geq -\tau(y - m) \geq -\tau(a - m)$. Hence the difference is at least $(a - m)[(1 - \tau)\mathbf{1}(y \leq m) - \tau\mathbf{1}(y > m)]$, and taking expectations gives $(a - m)[(1 - \tau)\mathbb{P}(Y \leq m) - \tau(1 - \mathbb{P}(Y \leq m))]$ $= (a - m)[\mathbb{P}(Y \leq m) - \tau] \geq 0$. The case $a < m$ is symmetric with $\mathbb{P}(Y \geq m) \geq 1 - \tau$. Taking $\tau = \frac{1}{2}$ recovers Proposition 2.2.5 up to a factor $\frac{1}{2}$. $\square$

Example 2.2.7 (Three losses, three answers, one distribution). A delivery company must promise a delivery time a (minutes) for a route whose actual

time Y is 100 with probability 0.5, 200 with probability 0.3, and 600 with probability 0.2 (occasional road closures).

- *Squared loss.* Report the mean: $0.5(100) + 0.3(200) + 0.2(600) = 230$. Check against $a = 200$: risk at 230 is $0.5(130)^2 + 0.3(30)^2 + 0.2(370)^2 = 8450 + 270 + 27380 = 36100$; risk at 200 is $0.5(100)^2 + 0 + 0.2(400)^2 = 37000$. The mean wins.
- *Absolute loss.* $\mathbb{P}(Y \leq 100) = 0.5$ and $\mathbb{P}(Y \geq 100) = 1$; also $\mathbb{P}(Y \leq 200) = 0.8$ and $\mathbb{P}(Y \geq 200) = 0.5$. Every $m \in [100, 200]$ is a median, and indeed the expected absolute loss is 130 for every a in that interval ($a = 100$: $0.3(100) + 0.2(500) = 130$; $a = 200$: $0.5(100) + 0.2(400) = 130$), and larger outside (at $a = 230$: $0.5(130) + 0.3(30) + 0.2(370) = 148$). Note that the mean 230 is *not* optimal here: the rare 600 pulls the mean but barely moves the median.
- *Asymmetric loss.* If being late costs 9 times more than being early, take $\tau = 0.9$: we need $\mathbb{P}(Y \leq m) \geq 0.9$ and $\mathbb{P}(Y \geq m) \geq 0.1$, which holds exactly for $m = 600$. Promise 600 minutes: the company prefers to disappoint nobody.

“The best prediction” has three different values. Nothing about the world changed; only the consequences did.

Decision loss	Optimal prediction	Typical use
$(y - a)^2$	conditional mean	symmetric errors
$ y - a $	conditional median	robust point prediction
asymmetric absolute	conditional quantile	unequal under/over cost

Classification: which class to declare

In classification the output belongs to one of C categories, and the conditional distribution gives the probability of each class, $\mathbb{P}(Y = c \mid X = x)$ for $c = 1, \dots, C$. These probabilities describe what can be predicted about the unknown class. A class *decision* requires an additional step, so in classification the distinction between prediction and decision is particularly clear:

$$\underbrace{\{\mathbb{P}(Y = c \mid X = x)\}_{c=1}^C}_{\text{predicted class probabilities}} \xrightarrow{\text{decision criterion}} \underbrace{f^*(x)}_{\text{class decision}}.$$

Theorem 2.2.8 (Zero-one loss selects the most probable class). *Let $\ell_{\text{dec}}(y, a) = \mathbf{1}(y \neq a)$ (every mistake costs 1, correct decisions cost 0). Then*

$$f^*(x) = \arg \max_c \mathbb{P}(Y = c \mid X = x),$$

the Bayes classifier, and its conditional risk is $1 - \max_c \mathbb{P}(Y = c \mid X = x)$.

Proof. For action a , the conditional risk is $\mathbb{E}[\mathbf{1}(Y \neq a) \mid X = x] = \mathbb{P}(Y \neq a \mid X = x) = 1 - \mathbb{P}(Y = a \mid X = x)$. Minimising $1 - \mathbb{P}(Y = a \mid X = x)$ over a is the same as maximising $\mathbb{P}(Y = a \mid X = x)$. $\square$

Reading the formula

$\mathbb{E}[1(Y \neq a) \mid X = x]$ is “the probability of being wrong if I say a .” With class probabilities $(0.5, 0.3, 0.2)$, saying class 1 is wrong with probability 0.5, class 2 with probability 0.7, class 3 with probability 0.8. Say class 1. The expected error rate is 0.5, and no decision does better; that residual is the problem’s own uncertainty.

Choosing the most probable class is optimal *only when all mistakes cost the same*. When they do not, Bayes decision theory still applies: choose the action with the smallest conditional expected cost, which need not be the most probable class.

Theorem 2.2.9 (Cost-sensitive threshold for a binary decision). *Consider a binary problem in which class 1 is an event to detect, $p(x) = \mathbb{P}(Y = 1 \mid X = x)$, correct decisions cost 0, a false positive (declaring 1 when the truth is 0) costs $C_{\text{FP}} > 0$ and a false negative (declaring 0 when the truth is 1) costs $C_{\text{FN}} > 0$. The Bayes rule declares positive if and only if*

$$p(x) > \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}}$$

(with either decision optimal at equality). Equal costs give the threshold $\frac{1}{2}$.

Proof. Deciding “positive” is wrong exactly when $Y = 0$, which has probability $1 - p(x)$, so its conditional expected cost is $C_{\text{FP}} [1 - p(x)]$. Deciding “negative” is wrong exactly when $Y = 1$, so its cost is $C_{\text{FN}} p(x)$. Positive is better when

$$\begin{aligned} C_{\text{FP}}(1 - p(x)) < C_{\text{FN}} p(x) &\iff C_{\text{FP}} < (C_{\text{FP}} + C_{\text{FN}}) p(x) \\ &\iff p(x) > \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}}. \end{aligned}$$

With $C_{\text{FP}} = C_{\text{FN}}$ the right side is $\frac{1}{2}$. □

Reading the formula

Two numbers to compare, nothing more.

- Cost of raising the alarm = $C_{\text{FP}} \times \mathbb{P}(\text{no event}) = C_{\text{FP}} (1 - p)$.
- Cost of staying quiet = $C_{\text{FN}} \times \mathbb{P}(\text{event}) = C_{\text{FN}} p$.

Cyberattack detection with $C_{\text{FN}} = 10$ (missed intrusion) and $C_{\text{FP}} = 1$ (an hour of investigation): threshold $1/(1 + 10) \approx 0.09$. At $p(x) = 0.2$: alarm costs $1 \times 0.8 = 0.8$, silence costs $10 \times 0.2 = 2$. Raise the alarm, even though an attack is *unlikely*. Expensive false negatives push the threshold down; expensive false positives push it up.

Example 2.2.10 (Where the same probability gives different decisions).

- *Cyberattack detection.* Missed attack: prolonged intrusion. False alarm: investigation time. Threshold well below 0.5; accept many false alarms.
- *Card fraud with amount-dependent cost.* Blocking a legitimate payment costs a fixed $C_{\text{FP}} = 5$ (customer annoyance). Missing fraud costs the amount A of the transaction: $C_{\text{FN}} = A$. The threshold is $5/(5 + A)$. For a 50 MAD purchase, block if $p > 0.09$; for 1000 MAD, block if $p > 0.005$.

The threshold depends on x through the amount: the decision criterion is more elaborate than one fixed number.

- *Court trial.* Wrongful conviction (false positive) is usually treated as far worse than wrongful acquittal. Threshold far above 0.5: “beyond reasonable doubt.”
- *Medical screening.* A missed disease is costly, an unnecessary follow-up test is cheap. Threshold below 0.5.

In each case the probability $p(x)$ is the same kind of object; only the loss differs. Probability estimation and decision policy are separate design choices.

Example 2.2.11 (Three classes and a cost table: the most probable class loses). A triage system estimates that a patient is healthy, mildly ill or severely ill with probabilities $(0.5, 0.3, 0.2)$. Three actions: discharge, monitor, treat. Costs (rows: truth; columns: action):

	discharge	monitor	treat
healthy	0	1	3
mild	4	0	2
severe	20	6	0

Conditional expected costs: discharge = $0.5(0) + 0.3(4) + 0.2(20) = 5.2$; monitor = $0.5(1) + 0.3(0) + 0.2(6) = 1.7$; treat = $0.5(3) + 0.3(2) + 0.2(0) = 2.1$. The Bayes decision is *monitor*, although “healthy” is the most probable state and would be the zero-one answer. The 20 in the corner (discharging a severe case) dominates the calculation.

Technique: Solving a Bayes decision problem by hand

1. Write the conditional probabilities of the outcomes at the given x as a row.
2. Write the loss table: one row per outcome, one column per action.
3. For each column, take the dot product with the probability row. That is the conditional risk of the action.
4. Pick the smallest column. For regression with a formula loss, use the shortcuts: squared $\rightarrow$ mean, absolute $\rightarrow$ median, τ -asymmetric $\rightarrow \tau$ -quantile.
5. For binary detection, skip the table: compare $C_{\text{FP}}(1 - p)$ with $C_{\text{FN}}p$, or use the threshold $C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$.

Classic problems

- $Y \mid x$ is 0 with probability 0.6 and 10 with probability 0.4. Best prediction under squared loss? Under absolute loss? Mean 4; median 0 (since $\mathbb{P}(Y \leq 0) = 0.6 \geq \frac{1}{2}$ and $\mathbb{P}(Y \geq 0) = 1$).
- $C_{\text{FN}} = 4 C_{\text{FP}}$. Threshold? $1/(1 + 4) = 0.2$.
- Bayes error rate with class probabilities $(0.45, 0.35, 0.20)$? $1 - 0.45 = 0.55$: no classifier can do better at this x .

- When is the zero-one Bayes classifier also optimal under unequal costs? Whenever the probabilities are so lopsided that the ranking of conditional risks does not change, e.g. $p(x) = 0.999$ for any moderate cost ratio. In general it is not.

Gotcha

- “Predict the most likely value” is the Bayes rule for zero-one loss only. Under squared loss the mean may be a value that never occurs (230 minutes in Example 2.2.7).
- The threshold formula has C_{FP} on top. Students often invert it: remember that expensive false negatives (C_{FN} large) must give a *small* threshold.
- The Bayes rule needs the true $\mathbb{P}(Y | X = x)$. In practice we only have estimates; Section 2.3 onward is about getting them.

Summary: Decisions with a known distribution

- Probabilities describe uncertainty; the loss describes consequences; together they determine the optimal action via $f^*(x) = \arg \min_a \mathbb{E}[\ell_{\text{dec}}(Y, a) | X = x]$.
- Squared loss $\rightarrow$ conditional mean (Theorem 2.2.4); absolute loss $\rightarrow$ median; asymmetric $\rightarrow$ quantile.
- Zero-one loss $\rightarrow$ most probable class (Theorem 2.2.8); unequal costs $\rightarrow$ threshold $C_{FP}/(C_{FP} + C_{FN})$ (Theorem 2.2.9), or a cost-table comparison in general.
- “Best prediction” has no meaning until the loss is specified.

2.3 From Known Distributions to Finite Data

What are we trying to get?

We never know $\mathbb{P}(X, Y)$. What we have is a finite pile of examples. We want the most direct possible way to imitate the Bayes rule from examples (look at nearby ones), to see exactly what “nearby” depends on, and to understand why this direct approach breaks down when inputs have many features.

The training sample

Definition 2.3.1 (Training dataset, i.i.d. assumption). A *training dataset* is a finite collection

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$$

of N realised input–output pairs. Superscripts index examples, subscripts index components: $x_j^{(i)}$ is feature j of example i . In the standard supervised setting, the pairs are modelled as *independent and identically distributed* (i.i.d.) draws from $\mathbb{P}(X, Y)$: one draw does not alter another, and all follow the same distribution.

Reading the formula

$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$ is a table with N rows. Row i holds the d features of example i and its label. With $N = 3$ houses and $d = 2$ features (area, rooms):

i	$x_1^{(i)}$ (area)	$x_2^{(i)}$ (rooms)	$y^{(i)}$ (price)
1	145	4	1.2
2	80	2	0.7
3	200	5	1.9

$x_1^{(2)} = 80$: the area of the second house. The i.i.d. assumption says the three rows are like three independent tickets drawn from the same urn. It can fail: consecutive days of a time series are dependent; houses in one street are dependent; and if the market changes, future draws come from a different urn (*distribution shift*).

The distinction is fundamental:

True distribution $\mathbb{P}(X, Y)$	Training data $\mathcal{D}$
unknown	observed finite sample

If the distribution were known, Definition 2.2.2 would give the optimal rule. Since we have only the sample, the task of learning is to use it to construct a rule that approximates what we would want under the unknown distribution. This move from a known distribution to finite observations is *the* central statistical step in supervised learning.

Estimating from nearby examples

The Bayes rules suggest a direct strategy. For regression we need the conditional mean at x . If many training examples had exactly the input x , we would average their outputs. Exact repetitions are rare when inputs contain continuous features, so we use examples whose representations are *close* to x .

To estimate the average price of houses “like this one,” average the prices of the training houses most like this one.

Definition 2.3.2 (k -nearest neighbours). Let $N_k(x)$ be the set of indices of the k training inputs nearest to x (for a chosen distance). The k NN regression estimate and the k NN class-probability estimate are

$$\hat{f}_k(x) = \frac{1}{k} \sum_{i \in N_k(x)} y^{(i)}, \quad \hat{p}_k(Y = c \mid X = x) = \frac{1}{k} \sum_{i \in N_k(x)} \mathbf{1}(y^{(i)} = c).$$

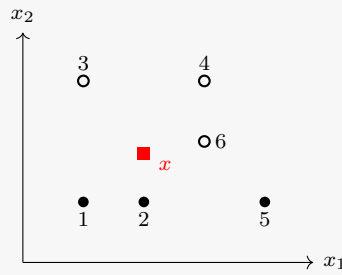
Under zero-one loss, the k NN classifier chooses $\arg \max_c \hat{p}_k(Y = c \mid X = x)$: the *majority vote* among the neighbours.

Reading the formula

$\sum_{i \in N_k(x)}$ means “add up over the k neighbours.” $\frac{1}{k} \sum y^{(i)}$ is their average label. $\frac{1}{k} \sum \mathbf{1}(y^{(i)} = c)$ counts how many neighbours have class c and divides by k : the *fraction* of neighbours in class c . If $k = 5$ and three neighbours are class B, then $\hat{p}_5(Y = B \mid x) = 3/5$, and majority vote says B.

Example 2.3.3 (kNN by hand). Six training points in the plane, each with a class and a numerical value:

i	$x_1^{(i)}$	$x_2^{(i)}$	class	$y^{(i)}$
1	1	1	A	10
2	2	1	A	12
3	1	3	B	20
4	3	3	B	22
5	4	1	A	14
6	3	2	B	18



(filled: class A; open: class B.) Query $x = (2, 1.8)$. Squared Euclidean distances: to point 1: $1 + 0.64 = 1.64$; point 2: $0 + 0.64 = 0.64$; point 3: $1 + 1.44 = 2.44$; point 4: $1 + 1.44 = 2.44$; point 5: $4 + 0.64 = 4.64$; point 6: $1 + 0.04 = 1.04$. Order of closeness: 2, 6, 1, {3, 4}, 5.

- $k = 1$: neighbour {2}. Class A; regression 12.
- $k = 3$: neighbours {2, 6, 1}, classes A, B, A. Estimated $\hat{p}(A) = 2/3$: class A; regression $(12 + 18 + 10)/3 = 13.3$.
- $k = 5$: neighbours {2, 6, 1, 3, 4}, classes A, B, A, B, B. $\hat{p}(B) = 3/5$: class B; regression $(12 + 18 + 10 + 20 + 22)/5 = 16.4$.

The decision flips from A to B as k grows: small k listens to the single closest point, large k averages over a wider region.

The meaning of “nearest” depends on the representation and on the distance used. The neighbourhood size k controls how local the estimate is: with small k the prediction follows individual observations and is sensitive to noise; with large k it averages over a broad region and can erase genuine local variation. k is a *hyperparameter*: it is chosen as part of model development (Section 2.6), not estimated by the averaging rule itself. Geometrically, for $k = 1$ the input space is partitioned into *Voronoi cells*: the cell of training point $x^{(i)}$ is the set of points closer to $x^{(i)}$ than to any other training point, and every query in that cell receives the label $y^{(i)}$.

Distance defines the neighbourhood

Definition 2.3.4 (Minkowski, Euclidean, Manhattan, Hamming distances). For $x, z \in \mathbb{R}^d$ and $p \geq 1$, the *Minkowski distance* is

$$d_p(x, z) = \left(\sum_{j=1}^d |x_j - z_j|^p \right)^{1/p}.$$

$p = 1$ gives the *Manhattan* distance (sum of coordinate gaps), $p = 2$ the *Euclidean* distance (straight-line length), and $p \rightarrow \infty$ the largest coordinate gap. For binary vectors, the *Hamming* distance counts the positions where they differ. A *scaled, weighted* Euclidean distance is

$$d_w(x, z) = \sqrt{\sum_{j=1}^d w_j \left(\frac{x_j - z_j}{s_j} \right)^2},$$

where s_j is a scale (units, spread) and $w_j \geq 0$ a weight ($w_j = 0$ ignores feature j).

Reading the formula

For $x = (1, 2)$ and $z = (3, 5)$: coordinate gaps 2 and 3. Manhattan $d_1 = 2 + 3 = 5$ (walk along the streets). Euclidean $d_2 = \sqrt{2^2 + 3^2} = \sqrt{13} \approx 3.61$ (as the crow flies). Largest gap $d_\infty = 3$. Hamming distance between 10110 and 11100: positions 2 and 4 differ, so 2. In d_w , dividing by s_j converts each feature to “how many typical spreads apart,” and w_j says how much that feature should count.

Example 2.3.5 (Scaling decides who the neighbours are). Two customers, described by (income in USD, age in years): $P = (30,000, 25)$ and $Q = (31,000, 60)$. Query: $(30,800, 25)$, a 25-year-old earning 30,800. Raw Euclidean distances: to P : $\sqrt{800^2 + 0^2} = 800$; to Q : $\sqrt{200^2 + 35^2} = \sqrt{41,225} \approx 203$. The nearest neighbour is Q , a 60-year-old, because income differences of a few hundred dollars dwarf age differences of decades *in these units*. Standardise with typical spreads $s_{\text{income}} = 1000$, $s_{\text{age}} = 10$: $P \rightarrow (30.0, 2.5)$, $Q \rightarrow (31.0, 6.0)$, query $\rightarrow (30.8, 2.5)$. Now to P : 0.8; to Q : $\sqrt{0.04 + 12.25} \approx 3.5$. The nearest neighbour is P , the customer of the same age. Nothing about the customers changed; the choice of units silently chose the answer. Standardising each feature, $z_j = (x_j - \mu_j)/\sigma_j$ with μ_j, σ_j its mean and standard deviation on the training data, is the usual remedy.

Gotcha

Compute μ_j and σ_j on the *training* data only and reuse them for new inputs. Recomputing them on the test data leaks information and makes evaluation optimistic (Section 2.6).

The curse of dimensionality

kNN needs enough observations near the query. Two ways to see that high-dimensional inputs defeat this.

Proposition 2.3.6 (Grid count). Covering $[0, 1]^d$ with a grid of 10 positions per coordinate requires 10^d locations. Going from $d = 2$ to $d = 10$ raises this

In high dimension, “nearby” points do not exist: everything is far from everything.

from 10^2 to 10^{10} .

Proof. Each of the d coordinates is chosen independently among 10 positions, giving $10 \times \cdots \times 10 = 10^d$ combinations. $\square$

Proposition 2.3.7 (Probability of finding a neighbour). *Let n points be uniformly distributed in the unit ball of $\mathbb{R}^d$ and let the query sit at its centre. Call a point a “true neighbour” if its distance to the query is below 0.1. The probability that a given point is a true neighbour is $p_d = (0.1)^d = 10^{-d}$, and the probability that at least one of the n points is a true neighbour is*

$$\mathbb{P}(\text{at least one neighbour}) = 1 - (1 - 10^{-d})^n.$$

Proof. A uniform point lands in a region with probability equal to the region’s volume divided by the ball’s volume. The ball of radius r in $\mathbb{R}^d$ has volume $V_d(r) = r^d V_d(1)$ (scaling every coordinate by r scales d -dimensional volume by r^d), so $p_d = V_d(0.1)/V_d(1) = 0.1^d$. The n points are independent, so the probability that *none* is a neighbour is $(1 - p_d)^n$, and the complement gives the claim. $\square$

Reading the formula

With $n = 1000$ points:

d	1	2	3	4	5	6	10
$\mathbb{P}(\geq 1 \text{ neighbour})$	$\approx 100\%$	99.996%	63.2%	9.5%	1.0%	0.1%	$10^{-5}\%$

For $d = 3$: $1 - (1 - 0.001)^{1000} \approx 1 - e^{-1} \approx 0.632$. For $d = 4$: $1 - (1 - 10^{-4})^{1000} \approx 1 - e^{-0.1} \approx 0.095$. With a thousand data points and ten features, the chance that *any* training point is genuinely close to the query is one in ten million. The “neighbours” kNN uses are then far away, and the local averaging idea loses its justification.

This does not mean every high-dimensional problem is hopeless. It shows that *local* estimation cannot exploit high-dimensional inputs efficiently without additional structure, and motivates a different strategy: describe how predictions vary across the input space through a *shared* model and learn its adjustable quantities from all training examples at once (Section 2.4).

Computation

kNN is a *lazy* method: nothing is computed at training time; all the work happens at prediction time. The basic implementation stores all N inputs, memory $O(Nd)$, and computes N distances per query, each costing $O(d)$, for $O(Nd)$ per prediction. Tree structures help in low dimension; approximate nearest-neighbour search and batched distance computations help at scale.

Technique: kNN in practice

- Standardise features first (Example 2.3.5); one-hot categorical features (Example 2.1.2).
- Choose k on validation data (Section 2.6); odd k avoids ties in binary classification.

- For a hand computation, use *squared* Euclidean distances: same ordering, no square roots.
- Distrust kNN when d is large relative to N (Proposition 2.3.7).

Classic problems

- In Example 2.3.3, what does $k = 6$ predict? All points: $\hat{p}(A) = 3/6$: a tie; regression $(10 + 12 + 20 + 22 + 14 + 18)/6 = 16$. With $k = N$ every query gets the same answer: the global average.
- $n = 1000$, $d = 3$, radius 0.1: *expected number of true neighbours?* $np_d = 1000 \times 10^{-3} = 1$.
- What is the 1-NN training error? Zero (each training point is its own nearest neighbour), which says nothing about generalization; recall the memoriser of Chapter 1.

Summary: From distribution to data

- Data $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$, modelled as i.i.d. draws from the unknown $\mathbb{P}(X, Y)$.
- kNN imitates the Bayes rule locally: average (regression) or vote (classification) over the k nearest training points.
- “Nearest” depends on the distance and on feature scaling; standardise.
- Curse of dimensionality: $\mathbb{P}(\text{a neighbour within } 0.1) = 1 - (1 - 10^{-d})^n$ collapses as d grows; local methods need shared structure to survive in high dimension.

2.4 Parameterized Models

What are we trying to get?

Since finite data cannot tell us what to do at every input, we restrict ourselves in advance to a *family* of candidate rules and let the data pick one member. We want to see what such a family looks like, what its “knobs” are, and which knobs are learned versus chosen by us.

Definition 2.4.1 (Parameterized model, hypothesis space). A *parameterized model* is a function $f_\theta(x)$ whose behaviour is controlled by a vector of adjustable numbers $\theta \in \Theta$, the *parameters*. The set of functions the model can represent,

$$\mathcal{H} = \{f_\theta : \theta \in \Theta\},$$

is the *hypothesis space*. The model *form* determines how parameters are shared across inputs; θ selects one member of the family. Choosing $\mathcal{H}$ introduces structure before any data are seen: an *inductive bias*.

Reading the formula

Think of f_θ as a machine with dials. θ is the dial settings; $\mathcal{H}$ is everything the machine can do across all settings. *Linear regression*: $f_\theta(x) = w^\top x + b$ with $\theta = (w, b)$. For $w = (2, -1)$, $b = 3$, and input $x = (1, 4)$: $f_\theta(x) = 2 \cdot 1 + (-1) \cdot 4 + 3 = 1$. Turning the dials to $w = (0, 1)$, $b = 0$ gives a different linear function, $f(x) = x_2$. Every function in $\mathcal{H}$ is a flat surface (a line in one dimension, a plane in two); no dial setting can produce a curve. That restriction is the inductive bias of linear regression.

Example 2.4.2 (Parameters versus model form). A polynomial model in one variable, $f_\theta(x) = \theta_0 + \theta_1 x + \dots + \theta_p x^p$, makes the distinction clear. Changing the coefficients $\theta_0, \dots, \theta_p$ selects another polynomial *of the same degree*: same form, different parameters. Changing the degree p changes the *family* being considered: with $p = 1$ only lines are available, with $p = 2$ parabolas as well. The coefficients are learned; the degree is a decision made before learning.

Definition 2.4.3 (Nested hypothesis spaces). Hypothesis spaces $\mathcal{H}_1 \subseteq \mathcal{H}_2$ are *nested* when every function of the first is also available in the second. Polynomials of degree $\leq p$ are nested in polynomials of degree $\leq p + 1$ (set $\theta_{p+1} = 0$).

Classification models output probabilities

Definition 2.4.4 (Probability simplex, probabilistic classifier). For C -class classification, a model estimates the vector of conditional class probabilities

$$\hat{p}_\theta(x) = [\hat{p}_\theta(Y = 1 | x), \dots, \hat{p}_\theta(Y = C | x)]^\top \in \Delta^{C-1},$$

where the *probability simplex* is

$$\Delta^{C-1} = \left\{ p \in \mathbb{R}^C : p_c \geq 0, \sum_{c=1}^C p_c = 1 \right\}.$$

A decision rule (Section 2.2) then converts this vector into an action.

Reading the formula

Δ^{C-1} is just “all lists of C nonnegative numbers adding to 1,” i.e. all legitimate probability assignments over C classes. For $C = 3$: $(0.5, 0.3, 0.2)$ is in it; $(0.5, 0.5, 0.5)$ is not (sums to 1.5); $(1.2, -0.2, 0)$ is not (negative entry). A common way to build such a model is to compute C real-valued *scores* $s_{\theta,c}(x)$ and pass them through the *softmax* map $p_c = e^{s_c} / \sum_{c'} e^{s_{c'}}$: scores $(2, 0, -1)$ become $(7.39, 1, 0.37)/8.76 \approx (0.84, 0.11, 0.04)$, automatically nonnegative and summing to one.

Parameters and hyperparameters

Definition 2.4.5 (Parameters, hyperparameters). *Parameters* are quantities learned from the training examples. *Hyperparameters* are choices made around that estimation: some define the hypothesis space (polynomial degree, number of layers), others control the learning procedure (number of neighbours k , step sizes).

model form + hyperparameters $\rightarrow \mathcal{H} \rightarrow \theta \rightarrow$ particular model.

Model	Parameters (learned)	Hyperparameters (chosen)
kNN	none	k , the distance, feature scales
Linear regression	w, b	which features to include
Polynomial regression	coefficients $\theta_0, \dots, \theta_p$	degree p
Neural network	all weights	layers, width, learning rate

Gotcha

kNN has hyperparameters but *no* parameters: nothing is fitted, the data themselves are the model. That is why it is called lazy, and why k cannot be “learned” by the averaging rule: with $k = 1$ the training error is always zero, so training data alone would always choose $k = 1$.

Summary: Parameterized models

- f_θ : a function with adjustable parameters; $\mathcal{H}$: all functions it can produce. Choosing $\mathcal{H}$ is choosing an inductive bias.
- Linear and polynomial regression; probabilistic classifiers output a vector in the simplex Δ^{C-1} .
- Parameters are learned; hyperparameters are chosen and define the family or the procedure.

2.5 Learning the Parameters from Data

What are we trying to get?

We have a family of candidate rules and a pile of examples. We want a principle that says which parameter values the data prefer, and we want to understand exactly what that principle can and cannot promise: it finds the best fit to the examples, which is not the same as the best rule for the world.

Two losses, two jobs

Learning a parameterized model requires a criterion for comparing parameter values. It is useful to distinguish two losses:

- ℓ_{train} : guides the learning of the model from data,
- ℓ_{dec} : guides the choice of an action from inferred information.

They can coincide, as when squared loss is used both to fit and to assess a regression. They can differ: a classifier may be *trained* with cross-entropy to estimate probabilities and then *decide* with zero-one loss or application costs.

Population risk and empirical risk

Definition 2.5.1 (Population risk, empirical risk, ERM). For parameters θ , the *population risk* is the average training loss over the whole distribution,

$$R(\theta) = \mathbb{E}_{(X,Y) \sim \mathbb{P}}[\ell_{\text{train}}(Y, f_{\theta}(X))],$$

and the *empirical (training) risk* is the average over the observed examples,

$$\hat{R}_{\text{train}}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_{\text{train}}(y^{(i)}, f_{\theta}(x^{(i)})).$$

Empirical risk minimisation (ERM) chooses parameters that make the training risk as small as possible:

$$\hat{\theta} \in \arg \min_{\theta \in \Theta} \hat{R}_{\text{train}}(\theta).$$

The best parameters *in the population sense* are $\theta_{\mathcal{H}}^* \in \arg \min_{\theta} R(\theta)$; their population risk may exceed the Bayes risk when $\mathcal{H}$ does not contain the Bayes rule.

Reading the formula

Three quantities, in increasing order of how much we can compute them.

- $R(\theta)$: “if I used dial setting θ on *every* house that will ever exist, what would my average loss be?” This is what we care about and can never compute, because it needs $\mathbb{P}$.
- $\hat{R}_{\text{train}}(\theta)$: “on the N houses in my table, what *was* my average loss?” Computable. With three examples whose squared errors under θ are 1, 4, 0: $\hat{R} = (1 + 4 + 0)/3 = 5/3$.
- $\hat{\theta}$: the dial setting with the smallest $\hat{R}_{\text{train}}$. The hat says “computed from this particular sample”; another sample would give another $\hat{\theta}$.

Key insight

ERM replaces an average over the unknown world by an average over the sample. Everything that can go wrong in learning is contained in the gap between R and $\hat{R}$, and the next two propositions say when that gap is harmless and when it is not.

Proposition 2.5.2 (For a fixed model, training risk is an honest estimate). *Fix θ (chosen without looking at the data). If the examples are i.i.d. from $\mathbb{P}$, then $\mathbb{E}[\hat{R}_{\text{train}}(\theta)] = R(\theta)$, and by the law of large numbers $\hat{R}_{\text{train}}(\theta) \rightarrow R(\theta)$ as $N \rightarrow \infty$.*

Proof. $\hat{R}_{\text{train}}(\theta)$ is the average of N terms $\ell_{\text{train}}(y^{(i)}, f_{\theta}(x^{(i)}))$, each an independent copy of the random variable $\ell_{\text{train}}(Y, f_{\theta}(X))$, whose mean is $R(\theta)$ by definition. The expectation of an average of terms with mean $R(\theta)$ is $R(\theta)$. The convergence statement is the law of large numbers for i.i.d. terms with finite mean. $\square$

Proposition 2.5.3 (After minimisation, training risk is optimistic). *Let $\hat{\theta}$ be the ERM solution. Then*

$$\mathbb{E}[\hat{R}_{\text{train}}(\hat{\theta})] \leq \min_{\theta} R(\theta) \leq \mathbb{E}[R(\hat{\theta})].$$

In words: the training risk of the fitted model underestimates, on average, both the best achievable population risk and the population risk of the fitted model itself.

Proof. Let θ^* minimise R . Since $\hat{\theta}$ minimises $\hat{R}_{\text{train}}$, $\hat{R}_{\text{train}}(\hat{\theta}) \leq \hat{R}_{\text{train}}(\theta^*)$ for every sample. Taking expectations and using Proposition 2.5.2 for the fixed θ^* : $\mathbb{E}[\hat{R}_{\text{train}}(\hat{\theta})] \leq \mathbb{E}[\hat{R}_{\text{train}}(\theta^*)] = R(\theta^*) = \min_{\theta} R(\theta)$. The second inequality holds because $R(\hat{\theta}) \geq \min_{\theta} R(\theta)$ for every sample. $\square$

Reading the formula

The fitted model has been *chosen because* it looks good on these particular examples. Part of its good training score is real, part is luck that will not recur. The proposition quantifies that “the training score is at least as flattering as the truth.” A small training risk confirms a good fit to the observed examples; it is only an (optimistic) estimate of performance on unseen ones.

Proposition 2.5.4 (Bigger families fit the training data at least as well). *If $\mathcal{H}_1 \subseteq \mathcal{H}_2$, then $\min_{f \in \mathcal{H}_2} \hat{R}_{\text{train}}(f) \leq \min_{f \in \mathcal{H}_1} \hat{R}_{\text{train}}(f)$. In particular, increasing the polynomial degree never increases the minimum training risk.*

Proof. The minimum over a larger set is at most the minimum over a subset, because every candidate of $\mathcal{H}_1$ is also a candidate in $\mathcal{H}_2$. $\square$

Proposition 2.5.5 (Enough flexibility fits anything). *Given N examples with distinct inputs $x^{(1)}, \dots, x^{(N)} \in \mathbb{R}$, there is a polynomial of degree at most $N - 1$ passing through all N points; its training risk under any loss with $\ell_{\text{train}}(y, y) = 0$ is zero.*

Proof. For each i , the polynomial $L_i(x) = \prod_{j \neq i} \frac{x - x^{(j)}}{x^{(i)} - x^{(j)}}$ has degree $N - 1$, equals 1 at $x^{(i)}$ and 0 at every other $x^{(j)}$ (the denominators are nonzero because the inputs are distinct). Then $f(x) = \sum_{i=1}^N y^{(i)} L_i(x)$ has degree at most $N - 1$ and $f(x^{(i)}) = y^{(i)}$ for all i , so every training loss term is $\ell_{\text{train}}(y^{(i)}, y^{(i)}) = 0$. $\square$

Interpretation & discussion

Propositions 2.5.4 and 2.5.5 together explain why the training risk is a *poor guide for choosing model complexity*: making the family more flexible (higher degree, smaller k in kNN) can only lower the training risk, all the way to zero, even after performance on new examples has started to deteriorate. Parameter estimation and hyperparameter selection therefore need different evidence: training data estimate parameters *inside* each fixed configuration; separate validation data compare the fitted configurations (Section 2.6).

Example 2.5.6 (ERM solved by hand: least squares in one variable). Some objectives have an analytical solution. Fit $f_{\theta}(x) = wx + b$ to the four points

$(0, 1), (1, 3), (2, 2), (3, 4)$ under squared loss. The training risk is $\hat{R}(w, b) = \frac{1}{4} \sum_i (y^{(i)} - wx^{(i)} - b)^2$. Setting both partial derivatives to zero,

$$\frac{\partial \hat{R}}{\partial b} = -\frac{2}{N} \sum_i (y^{(i)} - wx^{(i)} - b) = 0, \quad \frac{\partial \hat{R}}{\partial w} = -\frac{2}{N} \sum_i x^{(i)} (y^{(i)} - wx^{(i)} - b) = 0,$$

gives the *normal equations* $\sum y = w \sum x + Nb$ and $\sum xy = w \sum x^2 + b \sum x$, whose solution is

$$w = \frac{N \sum xy - \sum x \sum y}{N \sum x^2 - (\sum x)^2}, \quad b = \frac{\sum y - w \sum x}{N}.$$

Here $N = 4$, $\sum x = 6$, $\sum y = 10$, $\sum x^2 = 14$, $\sum xy = 0 + 3 + 4 + 12 = 19$. So $w = (76 - 60)/(56 - 36) = 0.8$ and $b = (10 - 4.8)/4 = 1.3$. Predictions 1.3, 2.1, 2.9, 3.7; residuals $-0.3, 0.9, -0.9, 0.3$; squared 0.09, 0.81, 0.81, 0.09; training risk $1.8/4 = 0.45$.

Nested families in action. Degree 0 (a constant): best constant is the mean 2.5, residuals $-1.5, 0.5, -0.5, 1.5$, risk $5/4 = 1.25$. Degree 1: 0.45. Degree 3: by Proposition 2.5.5 a cubic passes through all four points, risk 0. The sequence 1.25, 0.45, $\dots$, 0 is non-increasing, exactly as Proposition 2.5.4 says, and it says nothing about which degree predicts a fifth point best.

Learning class probabilities with cross-entropy

For classification, the natural decision loss is zero-one, $\ell_{01}(y, a) = \mathbf{1}(y \neq a)$, with decision $a(x) = \arg \max_c \hat{p}_{\theta, c}(x)$. As a *training* loss it is unsuitable. Direct minimisation would use

$$\hat{R}_{01}(\theta) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(y^{(i)} \neq \arg \max_c s_{\theta, c}(x^{(i)})),$$

which (i) discards changes in confidence, (ii) is piecewise constant in θ , and (iii) has zero gradient wherever it is defined and no gradient where the decision flips.

Example 2.5.7 (Why the zero-one training risk cannot be optimised smoothly). One feature, a threshold classifier $f_{\theta}(x) = 1$ if $x > \theta$, else 0. Data: $x = 1, 2, 3, 4$ with labels 0, 0, 1, 1. Then $\hat{R}_{01}(\theta)$ is $2/4$ for $\theta < 1$, $1/4$ for $1 \leq \theta < 2$, 0 for $2 \leq \theta < 3$, $1/4$ for $3 \leq \theta < 4$, $2/4$ for $\theta \geq 4$: a staircase. Between the steps the slope is exactly 0, so “move θ in the direction that decreases the loss” gets no information; at the steps the function jumps. Any method that improves by small steps is blind. A useful training loss must vary smoothly with the model output and reward better probability estimates.

Definition 2.5.8 (Cross-entropy training loss). Let $\hat{p}_{\theta, c}(x) = \hat{p}_{\theta}(Y = c \mid X = x)$ be the model’s probability for class c . For an example $(x^{(i)}, y^{(i)})$, the *cross-entropy loss* is

$$\ell_{\text{CE}}(y^{(i)}, \hat{p}_{\theta}(x^{(i)})) = -\log \hat{p}_{\theta, y^{(i)}}(x^{(i)}),$$

the negative log of the probability assigned to the class that actually occurred. The *empirical cross-entropy* is

$$\hat{R}_{\text{CE}}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log \hat{p}_{\theta, y^{(i)}}(x^{(i)}), \quad \hat{\theta}_{\text{CE}} \in \arg \min_{\theta} \hat{R}_{\text{CE}}(\theta).$$

Zero-one loss only notices which class won.

Cross-entropy notices how much probability the true class got, so it can guide small improvements.

For binary classification, writing $\hat{p} = \hat{p}_\theta(Y = 1 \mid x)$ and $y \in \{0, 1\}$, this is $\ell_{\text{BCE}}(y, \hat{p}) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$.

Reading the formula

$\hat{p}_{\theta, y^{(i)}}(x^{(i)})$ is a mouthful for a simple thing: “look up the probability the model gave to the *correct* class of example i .” Then take minus its log. Natural logs:

probability given to the true class	0.9	0.6	0.1	0.01
loss $-\ln(\cdot)$	0.105	0.511	2.303	4.605

Three examples with true-class probabilities 0.9, 0.6, 0.1 give $\hat{R}_{\text{CE}} = (0.105 + 0.511 + 2.303)/3 \approx 0.973$. The confident mistake (0.1) dominates. The binary formula is the same idea written for $y \in \{0, 1\}$: if $y = 1$ only the term $-\log \hat{p}$ survives; if $y = 0$ only $-\log(1 - \hat{p})$, the log-probability of the class that occurred.

Where does this formula come from? Encode the observed label of example i as a *one-hot* vector $t_c^{(i)} = \mathbf{1}(y^{(i)} = c)$: a probability vector that puts all its mass on the observed class. Its cross-entropy with the model’s vector, in the sense of Definition 1.8.1, is $-\sum_c t_c^{(i)} \log \hat{p}_{\theta, c}(x^{(i)}) = -\log \hat{p}_{\theta, y^{(i)}}(x^{(i)})$, because only the term $c = y^{(i)}$ survives. The one-hot vector represents the *observed* label, not the unknown true distribution; averaging over many examples is what estimates the population quantity, as the next theorem shows.

Theorem 2.5.9 (Cross-entropy is minimised by the true probabilities). *Let $p_c(x) = \mathbb{P}(Y = c \mid X = x)$ be the true conditional probabilities. At a fixed x , the conditional expected cross-entropy of the model’s vector $\hat{p}_\theta(x)$ is*

$$\mathbb{E}[-\log \hat{p}_{\theta, Y}(x) \mid X = x] = H(p(x)) + D_{\text{KL}}(p(x) \parallel \hat{p}_\theta(x)) \geq H(p(x)),$$

with equality iff $\hat{p}_\theta(x) = p(x)$. Consequently the population cross-entropy risk decomposes as

$$R_{\text{CE}}(\theta) = H(Y \mid X) + \mathbb{E}_X[D_{\text{KL}}(p(X) \parallel \hat{p}_\theta(X))],$$

where $H(Y \mid X) = \mathbb{E}_X[H(p(X))]$ is the conditional entropy, and R_{CE} is minimised, over all probability models, exactly by models that output the true conditional probabilities.

Proof. At fixed x , Y takes value c with probability $p_c(x)$, so $\mathbb{E}[-\log \hat{p}_{\theta, Y}(x) \mid X = x] = -\sum_c p_c(x) \log \hat{p}_{\theta, c}(x) = H(p(x), \hat{p}_\theta(x))$, the cross-entropy of Definition 1.8.1. By equation (1.1) this equals $H(p(x)) + D_{\text{KL}}(p(x) \parallel \hat{p}_\theta(x))$, and by Gibbs’ inequality (Theorem 1.8.2) the divergence is ≥ 0 with equality iff the two vectors coincide. Averaging over X gives the decomposition of R_{CE} ; the first term does not depend on θ and the second is ≥ 0 with equality iff $\hat{p}_\theta(x) = p(x)$ for (almost) every x . $\square$

Reading the formula

Two terms. $H(Y \mid X)$ is the uncertainty about the class that *remains even if you know x* : the problem’s own noise, which no model can remove. The divergence term is *your error*: how far your probabilities sit from the true ones,

averaged over inputs. Training with cross-entropy pushes only the second term down. Recall the numbers of Example 1.8.3: with truth (0.7, 0.3) the floor is 0.881 bits, a hedged (0.5, 0.5) pays 0.119 extra, an overconfident (0.9, 0.1) pays 0.222 extra.

Key insight

Cross-entropy is the unique combination of two facts: (i) the one-hot cross-entropy of an example is just $-\log(\text{probability of the observed class})$, which is computable from data; (ii) its expectation is $H + D_{\text{KL}}$, which is minimised at the truth (Gibbs). So minimising something we *can* compute drives the model toward something we *want*: calibrated probabilities, usable afterwards by any decision rule of Section 2.2.

Proposition 2.5.10 (Cross-entropy minimisation is maximum likelihood). *Minimising $\hat{R}_{\text{CE}}(\theta)$ is equivalent to maximising the conditional likelihood of the observed labels, $L(\theta) = \prod_{i=1}^N \hat{p}_{\theta, y^{(i)}}(x^{(i)})$.*

Proof. $\log L(\theta) = \sum_i \log \hat{p}_{\theta, y^{(i)}}(x^{(i)}) = -N \hat{R}_{\text{CE}}(\theta)$. Since $\log$ is increasing and $-N < 0$, maximising L is the same as minimising $\hat{R}_{\text{CE}}$. $\square$

Interpretation & discussion

Cross-entropy gives a graded assessment: predicting the observed class with probability 0.9 costs less than with 0.6, and a confident wrong prediction costs a lot. It is differentiable for positive probabilities, so with a smooth probability model gradient methods can optimise it (later in the course). Zero-one loss stays in the picture as the *decision* loss: train with cross-entropy, then decide with $\arg \max$ or with a cost-sensitive threshold. The probabilities are learned once and can serve several decision rules.

Learning objective and learning algorithm

Definition 2.5.1 specifies a *learning objective*: which parameter values are preferred. A *learning algorithm* specifies how suitable values are found: some objectives have closed-form solutions (Example 2.5.6); most are solved approximately by iterative optimisation, producing a sequence $\theta_0, \theta_1, \dots$. The complete sequence separates learning, inference and decision:

$$\mathcal{D} \xrightarrow{\ell_{\text{train}}} \text{learned model } f_{\hat{\theta}} \xrightarrow{\text{new } x} \text{inferred quantity} \xrightarrow{\ell_{\text{dec}}} \text{action}.$$

Technique: Working with risks

- To compute an empirical risk by hand: one loss per example, then average. Keep a column of losses; the mistakes are usually in the arithmetic, not the concept.
- Squared-loss ERM for a line: use the normal-equation formulas of Example 2.5.6; check that residuals sum to zero (they must, from $\partial \hat{R} / \partial b = 0$).
- Cross-entropy by hand: write the probability of the *true* class for each example, take $-\ln$, average. Never take the log of the largest probability by mistake.

- Training risk after fitting is optimistic (Proposition 2.5.3); never report it as the model's performance.

Classic problems

- *A model gives the true class probability 0.5 on every example. Empirical cross-entropy?* $-\ln 0.5 = 0.693$ nats, whatever N .
- *Degree-5 polynomial fitted to 6 points with distinct x : training risk?* Zero (Proposition 2.5.5).
- *Can a larger hypothesis space have larger minimum training risk?* No (Proposition 2.5.4). Can it have larger population risk for the fitted model? Yes, and that is the whole problem of overfitting.
- *Binary example, $y = 0$, model says $\hat{p} = 0.8$. BCE loss?* $-\ln(1 - 0.8) = 1.609$.

Summary: Learning from data

- Population risk $R(\theta)$ (what we want, unknowable) versus empirical risk $\hat{R}_{\text{train}}(\theta)$ (what we can compute). ERM minimises the latter.
- Training risk is honest for a fixed model (Proposition 2.5.2) and optimistic after fitting (Proposition 2.5.3); bigger families only lower it (Propositions 2.5.4, 2.5.5).
- Cross-entropy $-\log \hat{p}_{\theta,y}(x)$ is a smooth training loss whose expectation is $H(Y | X) + \mathbb{E}[D_{\text{KL}}]$, minimised by the true probabilities (Theorem 2.5.9); it equals maximum likelihood.
- Objective (which θ is preferred) and algorithm (how to find it) are separate.

2.6 Evaluating a Learned Model

What are we trying to get?

Since training risk flatters the fitted model, we need evidence from examples that did *not* shape it. We want a disciplined way to split the data so that parameters, hyperparameters and the final performance estimate each rest on information not already used, and a way to do this when data are scarce.

Three subsets, three roles

Definition 2.6.1 (Training, validation and test sets). The available data are partitioned into three subsets with different roles.

Subset	Main role
Training set $\mathcal{D}_{\text{train}}$	learn model parameters
Validation set $\mathcal{D}_{\text{val}}$	select models and hyperparameters
Test set $\mathcal{D}_{\text{test}}$	evaluate the final model, after all choices are fixed

Their sizes need not be equal; the defining difference is how information from

each subset is allowed to influence the learning procedure.

The training set enters parameter learning through ℓ_{train} . The validation set compares the fitted models and may select k in kNN, the degree of a polynomial, a representation, or a model family. Reusing validation results for many design decisions gradually adapts the development process to that set, so a third sample is reserved for the final assessment. The test set is used once, after parameters, hyperparameters and evaluation rules are fixed; consulting it repeatedly during development would turn it into another validation set and make its estimate optimistic, by exactly the mechanism of Proposition 2.5.3.

Every subset is informative about new data only until you use it to make a choice. Then it becomes part of “training” in disguise.

Proposition 2.6.2 (Held-out risk of a fixed model is honest). *If the model f was chosen without using the examples of a held-out set of size M drawn i.i.d. from $\mathbb{P}$, then the held-out empirical risk $\frac{1}{M} \sum_i \ell(y^{(i)}, f(x^{(i)}))$ has expectation equal to the population risk of f .*

Proof. Identical to Proposition 2.5.2: f is fixed relative to the held-out examples, so each term has mean equal to the population risk. $\square$

The catch is the word *fixed*. Once we select, among several candidates, the one with the lowest validation risk, that minimum is optimistic (Proposition 2.5.3 with the validation set playing the role of the training set and the candidates playing the role of Θ). The test set restores honesty because nothing was selected on it.

Selecting hyperparameters

Definition 2.6.3 (Hyperparameter selection by validation). Let λ denote a hyperparameter. For each candidate value, learn parameters on the training set,

$$\hat{\theta}_\lambda = \arg \min_{\theta} \hat{R}_{\text{train}}(\theta; \lambda),$$

then compare the fitted models on the validation set and keep the best,

$$\hat{\lambda} = \arg \min_{\lambda} \hat{R}_{\text{val}}(\hat{\theta}_\lambda; \lambda).$$

Reading the formula

Two nested “pick the best” operations, on two different datasets. *Inner*: for degree $\lambda = 2$, find the best parabola on the training points; for $\lambda = 3$, the best cubic; and so on. *Outer*: evaluate each of those fitted curves on the validation points and keep the degree whose fitted curve did best *there*. Concretely, suppose validation risks come out as

degree λ	0	1	2	3	5
training risk of $\hat{\theta}_\lambda$	1.25	0.45	0.40	0.10	0.00
validation risk of $\hat{\theta}_\lambda$	1.30	0.50	0.62	1.90	8.40

Training risk keeps falling (Proposition 2.5.4); validation risk is smallest at $\lambda = 1$. Choose the line, $\hat{\lambda} = 1$. The validation criterion may differ from ℓ_{train} when the application values something else (e.g. cost-weighted errors).

When data are limited: cross-validation

A fixed validation set leaves fewer observations for parameter learning and makes the selection depend on one particular split.

Definition 2.6.4 (K -fold cross-validation). Divide the development data (training plus validation) into K folds of roughly equal size. For each candidate λ and each fold $j = 1, \dots, K$, train on the other $K - 1$ folds and compute the validation risk $\hat{R}_{\text{val}}^{(j)}(\lambda)$ on fold j . The *cross-validation estimate* is the average

$$\hat{R}_{\text{CV}}(\lambda) = \frac{1}{K} \sum_{j=1}^K \hat{R}_{\text{val}}^{(j)}(\lambda),$$

and $\hat{\lambda}$ minimises it. Every development example serves as validation data exactly once. After selecting $\hat{\lambda}$, parameters are typically re-learned on all development data. The test set remains untouched until final evaluation. $K = N$ is *leave-one-out* cross-validation.

Reading the formula

Cut the deck into K piles. Round 1: hide pile 1, train on piles $2, \dots, K$, score on pile 1. Round 2: hide pile 2, and so on. Average the K scores. With $K = 3$ and candidates $k \in \{1, 3\}$ for kNN, suppose the fold scores are

	fold 1	fold 2	fold 3	$\hat{R}_{\text{CV}}$
$k = 1$	0.40	0.20	0.60	0.40
$k = 3$	0.30	0.30	0.30	0.30

Choose $k = 3$. Notice that a single split using fold 2 alone would have chosen $k = 1$; averaging over folds protects against such accidents.

Technique: Splitting data without fooling yourself

- Decide the split *before* looking at any result. Typical: 60/20/20 or, with cross-validation, 80/20 development/test.
- Anything computed from data (feature scaling constants, vocabulary, imputed values) is computed on the training portion and *applied* to the others. Otherwise information leaks.
- Time series: split by time (train on the past, validate on the future); random splits leak the future into the past.
- Grouped data (several images of the same patient): keep a group entirely in one subset; otherwise the model is tested on patients it has seen.
- Touch the test set once. If you go back and change something afterwards, you need a new test set.

Gotcha

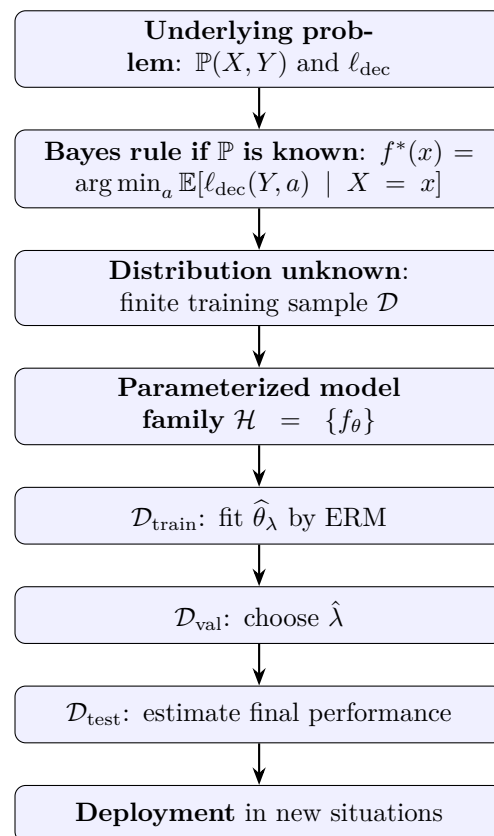
- “I chose k by minimising the validation error, so the validation error is my performance estimate.” No: that minimum is optimistic (Proposition 2.5.3). Report the test error.

- Cross-validation selects hyperparameters; it does not by itself produce a final model. Refit on all development data with $\hat{\lambda}$.
- A validation set with 10 examples cannot distinguish a 12% error rate from an 18% one. Small held-out sets give noisy verdicts.

Summary: Evaluation

- Training data learn θ ; validation data choose λ ; test data estimate final performance once.
- Held-out risk is honest for a fixed model (Proposition 2.6.2) and optimistic after selecting among candidates; hence the third set.
- K -fold cross-validation averages K validation risks so that every example validates once; refit afterwards.

2.7 The Learning Problem in One Picture



The picture also clarifies the relationship with Chapter 1. Experience is represented by finite data; the hypothesis space specifies which rules can be considered; the loss measures the quality of their outputs. Training risk guides parameter estimation, validation performance guides model complexity, and test performance estimates how the completed procedure may behave on new observations. The four ingredients of Definition 1.6.1 appear as $\mathcal{D}$ (experience), $\mathcal{H}$ (model), ERM plus an optimiser (learning algorithm), and ℓ_{dec} evaluated on held-out data (evaluation criterion). Why finite samples provide reliable evidence about future performance is the subject of generalization, developed in Lecture 3.

Example 2.7.1 (The whole pipeline on one problem). *Fraud detection.* Underlying problem: transactions X and fraud indicator Y ; decision loss: $C_{\text{FP}} = 5$ per blocked legitimate payment, $C_{\text{FN}} = \text{amount}$. Bayes rule: block iff $p(x) > 5/(5 + \text{amount})$; unknown, since p is unknown. Data: two years of labelled transactions, split by time into development (first 20 months) and test (last 4 months). Model family: a probabilistic classifier producing $\hat{p}_\theta(x)$, with hyperparameters λ (representation, model size). Training: minimise empirical cross-entropy on $\mathcal{D}_{\text{train}}$ for each λ . Validation: choose λ by cost-weighted error on $\mathcal{D}_{\text{val}}$ (not by cross-entropy, because the application cares about money). Test: apply the Bayes threshold to $\hat{p}_\theta$ on the final four months and report the total cost. Deployment: the threshold stays adjustable, because the decision policy is separate from the learned probabilities.

Notes on the original slides and notes

All formulas and numbers in the Lecture 2 material were rederived. The following points are worth attention.

- **“Mathematical proof” and “Proof” placeholder slides (12 and 17).** The slides leave the proofs of “squared loss $\rightarrow$ conditional mean” and “zero-one loss $\rightarrow$ most probable class” to the lecture. They are Theorems 2.2.4 and 2.2.8 here, together with the median and quantile cases the slides only tabulate.
- **Existence and uniqueness in the Bayes rule.** The notes write $f^*(x) \in \arg \min_a$ without conditions. The squared-loss result needs $\mathbb{E}[Y^2 \mid X = x] < \infty$; the median is generally *not* unique (Example 2.2.7: every value in $[100, 200]$); the argmax can tie. The “ $\in$ ” in the notes is the right symbol; treat “the” Bayes rule as “a” Bayes rule.
- **Curse of dimensionality (slide 36).** The computation places the query at the centre of the ball; near the boundary the neighbourhood is partly outside the ball and the probability is smaller still, so the table is if anything optimistic. The notes’ grid argument (10^d cells) and the slides’ ball argument (10^{-d} per point) are two views of the same phenomenon (Propositions 2.3.6 and 2.3.7).
- **Slide 26 (kNN motivation).** “emprical” is a typo for “empirical”; “ $\text{Ave}(Y \mid X \in N(x))$ ” is the same as the average in Definition 2.3.2.
- **Slide 14.** “Best prediction/prediction” is a duplicated word; the intended key point is “best prediction has no meaning until the loss is specified.”
- **Cross-entropy and the true probabilities (slide 55, notes Section 4.2).** “The expected loss is minimised at $\hat{p} = p$ ” is exactly right *over all probability vectors*; when the model family $\mathcal{H}$ cannot represent p , the minimiser within $\mathcal{H}$ is the member closest to p in the averaged KL sense of Theorem 2.5.9, which need not equal p .
- **Training risk as a guide to complexity (notes Section 5).** The notes assert that a more flexible family “can only improve or leave unchanged” the training fit and that some of the improvement is accidental. Propositions 2.5.4, 2.5.5 and 2.5.3 are the proofs.
- **Softmax.** The slides mention class scores $s_{\theta,c}$ on slide 50 without saying how scores become probabilities. The softmax map in the box after Defini-

tion 2.4.4 is the standard answer; it is not examinable from this lecture but makes the slide readable.

- **Slide 36 table**, $d = 1$. “ $\approx 100\%$ ” is $1 - 0.9^{1000} \approx 1 - 1.7 \times 10^{-46}$; correct.

Chapter takeaways

Summary: From representations to decisions

- **Representation.** Objects become vectors $x \in \mathbb{R}^d$ by human choices; one-hot code categories; the representation bounds what can be learned.
- **Known distribution.** The Bayes rule $f^*(x) = \arg \min_a \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x]$ is optimal (Theorem 2.2.3); squared loss gives the conditional mean, absolute the median, zero-one the most probable class, unequal costs the threshold $C_{\text{FP}} / (C_{\text{FP}} + C_{\text{FN}})$.
- **Finite data.** kNN imitates the Bayes rule locally; distance and scaling define “near”; the curse of dimensionality $(1 - (1 - 10^{-d})^n)$ kills locality in high dimension.
- **Models.** A hypothesis space $\mathcal{H} = \{f_\theta\}$ shares structure across inputs; parameters are learned, hyperparameters chosen.
- **ERM.** Minimise empirical risk; it is honest for fixed models and optimistic after fitting; cross-entropy trains calibrated probabilities because its expectation is $H(Y \mid X) + \mathbb{E}[\text{D}_{\text{KL}}]$.
- **Evaluation.** Train / validate / test, or cross-validate; touch the test set once.